

[Secure Exit](#)

CO3-AT1-Debugging and Optimization

[← Back](#)

SIMATS Online Programming Lab

JAVA PROGRAMMING

Akileshwaran A
192511354RD

Handling Debugging
10 marks

Q2

Hospital Registration –
NullPointerException
Debugging
10 marks

Q3

E-Commerce Product Search –
Array Exception Debugging
10 marks

Q4

Railway Reservation –
Synchronization Debugging
10 marks

Q5

Bank Account – Race Condition
Debugging

₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {  
  
    int balance = 10000;  
  
    void withdraw(int amount) {  
  
        if (balance >= amount) {
```

10 marks

5/5 answered

```
System.out.println(
Thread.currentThread().getName()
+ " withdrawing " + amount);

try {
Thread.sleep(100);
}
catch (InterruptedException e) {
System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
Thread.currentThread().getName()
+ " withdrawal successful");
}
else {
System.out.println(
Thread.currentThread().getName()
+ " insufficient balance");
}
}
}

class ATM extends Thread {

BankAccount account;
int amount;

ATM(BankAccount account, int amount) {
this.account = account;
```

```
this.amount = amount;
}

public void run() {
    account.withdraw(amount);
}
}

class BankApplication {

    public static void main(String[] args)
        throws InterruptedException {

        BankAccount account = new BankAccount();

        ATM atm1 = new ATM(account, 7000);
        ATM atm2 = new ATM(account, 6000);

        atm1.setName("ATM-1");
        atm2.setName("ATM-2");

        atm1.start();
        atm2.start();

        atm1.join();
        atm2.join();

        System.out.println(
            "Final Balance = " + account.balance);
    }
}

Correct the program so that only one withdrawal succeeds.
```

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 public class Main{
2     public static void main(String args[])throws InterruptedException{
3         ba a=new ba();
4         atm atm1=new atm(a,7000);
5         atm atm2=new atm(a,6000);
6         atm1.setName("ATM-1");
7         atm2.setName("ATM-2");
8         atm1.start();
9         atm2.start();
10        atm1.join();
11        atm2.join();
12        System.out.println("final balance =" + a.balance);
13    }
14 }
15 class ba{
16     int balance=10000;
17     synchronized void withdraw(int amount){
18         if(balance>=amount){
19             System.out.println(Thread.currentThread().getName()+" withdr
20             try{
21                 Thread.sleep(100);
22             }
23             catch(InterruptedException e){
24                 System.out.println("Interrupted");
25             }
26             balance =balance-amount;
27             System.out.println(Thread.currentThread().getName()+" withdr
28         }
29         else
```

```
30         System.out.println(Thread.currentThread().getName()+" Insuff
31     }
32 }
33 class atm extends Thread{
34     ba a;
35     int amount;
36     atm(ba a,int amount){
37         this.a=a;
38         this.amount=amount;
39     }
40     public void run(){
41         a.withdraw(amount);
42     }
43 }
```



Test Input

stdin input (optional)



Output

```
ATM-1 withdrawal 7000
ATM-1 withdrawal Successful
ATM-2 Insufficient Balance
final balance =3000
```

 Pending manual review

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.
